Use Case Name: Create Post

Element	Description
Use Case Name:	**Create Post**
Scenario:	User shares new content with their network.
Triggering Event:	User taps the **'Create New Post'** button.
Brief Description:	The User accesses the content creation interface, inputs content (text and/or media), confirms audience settings, and submits the post to be published to the feed.
Actors:	**User** (Primary Actor)
Related Use Cases:	Includes: *Upload Media*, *Define Audience/Privacy Settings*.
Stakeholders:	**Moderation/Safety Team:** To ensure content meets community guidelines. **Feed Service:** To distribute the content to followers/friends. **Analytics Team:** To track content creation patterns.
Preconditions:	User must be **logged in**. The post editor must be available.

Postconditions:	A new **Post Record** must be created in the database. Media (if applicable) must be successfully stored. The post is available for distribution to the feeds of the intended audience.

Flow of Activities:	Actor (User)	System
	1. User accesses the post editor screen.	**2.1** Display a blank post editor interface.
	3. User inputs content (text).	
	4. User optionally selects and adds media (photo/video).	**5.1** Initiate media upload process.
	6. User selects audience/privacy settings.	
	7. User taps the **'Post'** or **'Share'** button.	**8.1** Validate the content and audience settings.

		9.1 Create a new Post ID and save the post data.
		9.2 Push the new Post ID to the relevant Feed distribution queues.
Exception Conditions:	**5.1** If media upload fails (e.g., file too large, connection lost), then: a. System notifies the user of the failure. b. User can retry the upload or remove the media. **8.1** If content validation fails (e.g., empty post, prohibited words), then: a. System displays an error message explaining the rule violation. b. User must modify the content before resubmitting.	

Second Use Case : Comment On A Post

Element	Description
Use Case Name:	**Comment on Post**

Scenario:	**User submits a text reply or reaction to an existing post.**
Triggering Event:	**User taps the "Comment" or "Reply" button on a post.**
Brief Description:	**The User types their comment, and upon submission, the system records the comment, links it to the original post, increments the comment count, and notifies the relevant users (post author).**
Actors:	**User (Primary Actor)**
Related Use Cases:	**Includes: *View Post*, *Notify User*.**
Stakeholders:	**Post Author (needs engagement alert). Moderation Team (to enforce guidelines). Analytics Team (to track engagement).**
Preconditions:	**User must be logged in. The targeted post must exist and be visible to the commenting User.**

Postconditions:	**A new Comment Record must be created in the database. The original post's comment count is incremented. The comment is visible to the intended audience.**

Flow of Activities:	Actor (User)	System
	1. User taps the comment icon or field on a post.	2.1 Display the comment input interface (e.g., a text box).
	3. User types the desired comment text.	
	4. User taps the 'Send' or 'Post' button to submit the comment.	5.1 Validate the comment text (e.g., length, formatting, content).

		6.1 Create a new Comment ID and save the comment data.
		7.1 Update the comment count on the original post record.
		8.1 Trigger a notification for the post author and relevant users.
Exception Conditions:	5.1 If content validation fails (e.g., empty comment, malicious content), then: a. System displays an error message explaining the rule violation. b. User must modify the text before resubmitting. 6.1 If the original post is deleted just before submission, then: a. System returns an error: "Post not	

	found." b. Comment is discarded.	

Third Use Case : Like A Post

Element	Description
Use Case Name:	Like a Post
Scenario:	User registers approval for an existing post.
Triggering Event:	User taps the "Like" button (e.g., a heart icon) on a post.
Brief Description:	The User taps the icon, and the system instantly records the like, updates the visual status of the button, increments the total count, and alerts the post's author.
Actors:	User (Primary Actor)

Related Use Cases:	**Includes: *View Post*, *Notify User*.**
Stakeholders:	**Post Author (receives engagement alert). Analytics Team (tracks engagement metrics). Feed Ranking Service (uses likes for content sorting).**
Preconditions:	**User must be logged in. The targeted post must exist and be visible to the User. The User must not have previously liked the post (for the normal flow).**
Postconditions:	**A new Like Record must be created in the database. The post's like count is incremented. The like button changes its visual state (e.g., to filled).**

Flow of Activities:	**Actor (User)**	**System**
	1. User taps the 'Like' icon on a post.	**2.1 System validates the request (User is logged in,**

		Post exists).
		3.1 System creates a new Like Record, associating the User ID and Post ID.
		4.1 System updates the Post Database to increment the like counter.
		5.1 System changes the visual state of the 'Like' icon on the User's screen.
		6.1 System triggers a notification for the post

		author.
Exception Conditions:	2.1 If the User has already liked the post, then the system pauses this use case and invokes an "Unlike" action (deleting the record and decrementing the count). 3.1 If the post no longer exists, then: a. System returns an error: "Post not found." b. The client is prompted to refresh the feed.	

4th Use Case:Search A User

Element	Description
Use Case Name:	Search for a User
Scenario:	User attempts to locate another profile on the platform.
Triggering Event:	User enters text into the search bar and/or presses the 'Search' key.

Brief Description:	**The User submits a query (e.g., a username or name). The system queries the user directory, filters the results based on relevance, and displays the matching profiles to the User.**
Actors:	**User (Primary Actor)**
Related Use Cases:	**Includes: *View User Profile*.**
Stakeholders:	**User Database/Index Service (provides query results). Privacy Service (ensures only publicly searchable profiles are returned).**
Preconditions:	**User must be logged in. The search service must be operational.**
Postconditions:	**A list of matching User Profiles is displayed to the User. The search query may be logged for analytics or personalized suggestions.**

Flow of Activities:	**Actor (User)**	**System**

	1. User navigates to the search interface.	2.1 Display the search bar and recent/suggested searches.
	3. User enters a search query (e.g., a name or username).	
	4. User taps the 'Search' button or selects an auto-suggestion.	5.1 System validates the query (e.g., minimum length).
		6.1 System queries the User Index Service using the input.
		7.1 System filters the results based on privacy settings and relevance.
		8.1 Display the list of matching User Profiles to

		the Actor.
Exception Conditions :	5.1 If the search query is empty or too short, then: a. System prevents the search submission. b. System displays a message to the user ("Please enter at least 3 characters"). 7.1 If no users match the search query, then: a. System returns an empty result set. b. System displays a message to the user ("No users found matching your search").	

5th Use Case : Add Friend

Element	Description
Use Case Name:	Add Friend
Scenario:	A User initiates a connection with another User on the platform.

Triggering Event:	**User taps the "Add Friend" or "Follow" button on another User's profile.**
Brief Description:	**The system checks the relationship status and the other User's privacy settings. It then either instantly creates a 'Follow' relationship or generates a 'Pending' Friend Request and sends a notification.**
Actors:	**User (Initiator)**
Related Use Cases:	**Includes: *View User Profile*, *Notify User*, *Accept Friend Request*.**
Stakeholders:	**Relationship Service (to manage connections). Notification Service (to alert the recipient). Privacy Service (to enforce connection rules).**
Preconditions:	**User must be logged in. The target User must exist. The Users must not already be friends/followers.**
Postconditions:	**A new Relationship Record is created in the database with a status of 'Pending' or 'Accepted'. A notification is sent to the target User.**

Flow of Activities:	Actor (User)	System
	1. User navigates to the target User's profile.	
	2. User taps the "Add Friend" or "Follow" button.	3.1 System validates the request and checks the target User's privacy settings.
		4.1 If target settings are Public (Follow): System immediately creates an 'Accepted' Relationship Record.

		5.1 If target settings are Private (Friend Request): System creates a 'Pending' Relationship Record.
		6.1 System triggers a notification to alert the target User of the request/follow.
		7.1 System changes the visual state of the button on the Initiator's screen (e.g., to "Request Sent" or "Following").

Exception Conditions:	3.1 If the Users are already friends, then: a. System displays a message: "Already connected." b. Button state remains "Friends." 4.1 If the target User has blocked the Initiator, then: a. System silently fails to create the record. b. Button state remains unchanged or disappears. 5.1 If the target User is a system-controlled account (e.g., bot) and cannot be followed, then: a. System displays an error: "Cannot connect to this account."	

6th Use Case : Edit Post

Element	Description
Use Case Name:	Update Post
Scenario:	A User modifies the content of a post they previously published.

Triggering Event:	**User taps the 'Edit Post' option on one of their own posts.**
Brief Description:	**The system retrieves the existing post data, allows the User to modify text and/or media, and then saves the revised content, updating the timestamp (optional) and notifying the feed service.**
Actors:	**User (Post Author)**
Related Use Cases:	**Includes: *View Post*, *Create Post* (for accessing the editor).**
Stakeholders:	**Post Database (to store the revision). Moderation/Safety Team (to re-validate content). Feed Service (to refresh the post in feeds).**
Preconditions:	**User must be logged in. User must be the original author of the post. The post must exist and be eligible for editing (some platforms have time limits).**
Postconditions:	**The original Post Record is updated with the new content. A revision history may be logged.**

	The update is visible in the feeds.

Flow of Activities:	Actor (User)	System
	1. User selects the 'Edit Post' option on their existing post.	2.1 System validates the User's permission and post edit eligibility.
		3.1 System retrieves the existing content and displays it in the editor interface.
	4. User modifies the text and/or media attached to the post.	

	5. User taps the 'Save' or 'Done' button to submit the changes.	6.1 System validates the modified content against community rules.
		7.1 System updates the original Post Record with the new content and a 'Last Edited' timestamp.
		8.1 System pushes an update notification to the Feed Service to refresh the content for the audience.

Exception Conditions:	2.1 If the post is too old (past the platform's edit limit), then: a. System displays an error: "This post can no longer be edited." b. Editing interface is closed. 6.1 If the modified content fails validation (e.g., prohibited material), then: a. System rejects the save and displays a warning. b. User must revert or correct the change. 7.1 If the post has been reported or is under moderation review, then: a	

7th Use Case :Delete Post

Element	Description
Use Case Name:	Delete Post
Scenario:	A User removes their own previously published post from the platform.
Triggering Event:	User selects the 'Delete' option on one of their posts and confirms the action.

Brief Description:	**The system verifies the User is the author of the post. Upon confirmation, the post and all related interactions (comments, likes) are removed from the database and distribution queues.**
Actors:	**User (Primary Actor)**
Related Use Cases:	**Includes: *View Post*, *Remove Related Interactions*.**
Stakeholders:	**Post Database (primary data store for deletion). Content Moderation Logs (may need to log the deletion). Feed Service (needs to remove the content).**
Preconditions:	**User must be logged in. User must be the author of the post. The post must exist in the database.**
Postconditions:	**The Post Record is permanently removed or soft-deleted from the database. All associated Comments and Likes are removed. The post is removed from all distribution feeds.**

Flow of Activities:	Actor (User)	System
	1. User accesses the menu for a post they authored.	
	2. User selects the 'Delete Post' option.	3.1 System displays a confirmation prompt to the User.
	4. User confirms the deletion.	5.1 System validates that the User is the post author (authorization check).
		6.1 System initiates the deletion of all associated data (Comments, Likes).

		7.1 System removes or soft-deletes the main Post Record from the database.
		8.1 System sends a command to the Feed Service to remove the post from all distributed feeds.
	9. User is redirected to their profile or the main feed.	10.1 System displays a confirmation message ("Post successfully deleted").

Exception Conditions:	5.1 If the User is not the author of the post, then: a. System rejects the request (Authorization failure). b. System displays an error message ("You do not have permission to delete this post"). 7.1 If the Post ID cannot be found in the database during deletion, then: a. System logs the error but proceeds to confirm deletion (as the post is effectively gone). b. System displays a message: "Post deletion confirmed (or already deleted)."	

Delete Comment Use Case:

Element	Description
Use Case Name:	Delete Comment
Scenario:	A User removes a comment they or another authorized user (e.g., a moderator or post author) previously left on a post.

Triggering Event:	**User selects the 'Delete' option on a comment and confirms the action.**
Brief Description:	**The system verifies the User's authorization (must be the comment author or post author). If authorized, the comment record is removed from the database, and the comment count on the parent post is decremented.**
Actors:	**User (Comment Author or Post Author)**
Related Use Cases:	**Includes: *View Post*, *Decrement Comment Count*.**
Stakeholders:	**Comment Database (primary data store for deletion). Post Database (needs count update).**
Preconditions:	**User must be logged in. The comment must exist. User must be authorized to delete the comment.**
Postconditions:	**The Comment Record is removed or soft-deleted. The comment count on the parent post is decremented.**

Flow of Activities:	Actor (User)	System
	1. User accesses the menu for the comment.	
	2. User selects the 'Delete Comment' option.	3.1 System displays a confirmation prompt to the User.
	4. User confirms the deletion.	5.1 System validates User authorization (Is User comment author or post author?).
		6.1 System retrieves the parent Post ID associated with the comment.

		7.1 System removes or soft-deletes the Comment Record.
		8.1 System updates the Post Database to decrement the comment count.
	9. User observes the comment disappear.	10.1 System displays a brief confirmation ("Comment deleted").
Exception Conditions:	5.1 If the User is not authorized (neither comment author nor post author), then: a. System rejects the request (Authorization failure). b. System displays an error message ("You do not have permission to delete this comment"). 7.1 If the Comment ID cannot be found in the database during deletion,	

	then: a. System logs the error but confirms deletion (as the comment is effectively gone).	

Update Comment Use Case:

Element	Description
Use Case Name:	Update Comment
Scenario:	A Comment Author wishes to edit the text of an existing comment.
Triggering Event:	User selects the 'Edit' option on a comment they authored.
Brief Description:	The system presents the existing comment text in an editable field. The User makes changes and saves the update. The system validates the new text and updates the Comment Record, often storing an "Edited" timestamp.

Actors:	**User (Comment Author)**
Related Use Cases:	**Includes: *View Post*.**
Stakeholders:	**Comment Database (receives update). Moderation Team (may review edited content).**
Preconditions:	**User must be logged in and be the author of the comment. The comment must exist.**
Postconditions:	**The Comment Record in the database is updated with the new text. An "Edited" timestamp is applied.**

Flow of Activities:	**Actor (User)**	**System**
	1. User accesses the menu for a comment they authored.	
	2. User selects the 'Edit Comment' option.	**3.1 System validates that the User is the**

		author.
		4.1 System displays the existing comment text in an editable field.
	5. User modifies the comment text.	
	6. User taps the 'Save' or 'Update' button.	7.1 System validates the new comment text (e.g., against content rules).
		8.1 System updates the Comment Record in the database with the new text.

		9.1 System applies an "Edited" timestamp to the record.
	10. User observes the updated comment.	11.1 System displays a confirmation message ("Comment updated").
Exception Conditions:	3.1 If the User is not the author of the comment, then: a. System rejects the request (Authorization failure). b. System displays an error message ("You can only edit your own comments"). 7.1 If the new comment text fails validation (e.g., empty or malicious), then: a. System displays an error explaining the rule violation. b. User must modify the text before resubmitting.	